Analysis and Observations

Analysis and Observations

1. Artificial Neural Network Implementation

A simple feed-forward Artificial Neural Network (ANN) was implemented from scratch using NumPy. The network consists of an input layer, one hidden layer, and an output layer. Backpropagation was used to compute gradients of the loss function with respect to the network weights, and gradient descent was used to update the parameters.

The network architecture used was:

- XOR problem: 2–2–1 (2 inputs, 2 hidden neurons, 1 output)
- Cosine approximation: 1–2–1 (1 input, 2 hidden neurons, 1 output)

Activation functions were chosen based on the nature of the task. The **tanh** activation function was used in the hidden layer because it introduces nonlinearity and has zero-centered outputs, which helps optimization. For the output layer, **sigmoid** was used for the XOR classification problem, while a **linear activation** was used for the cosine regression task.

The loss function used for both problems was **Mean Squared Error (MSE)**.

2. XOR Function Learning

The XOR function is a classical example of a non-linearly separable problem. A single-layer perceptron cannot solve XOR because the classes cannot be separated by a linear decision boundary. By introducing a hidden layer with nonlinear activation functions, the network can learn more complex decision boundaries.

The network was trained on XOR inputs with a small amount of Gaussian noise added to the input values. This makes the learning problem slightly more realistic and prevents the model from memorizing exact binary patterns.

The training results show that the network is capable of learning the XOR mapping. In some configurations, particularly when stochastic gradient descent (batch size = 1) was used, the network achieved near-perfect or perfect classification accuracy. However, with larger batch sizes or deterministic gradient descent, the model sometimes converged to suboptimal solutions with slightly lower accuracy.

This behavior occurs because stochastic gradient descent introduces noise in the gradient updates, which helps the optimization process escape poor local minima. Deterministic

gradient descent follows smoother gradients and can sometimes converge more slowly or get stuck in less optimal regions.

3. Cosine Function Approximation

The second task was to train the neural network to approximate the cosine function. Unlike the XOR problem, cosine approximation is a regression task where the network learns a continuous mapping from input values to output values.

The input to the network is a scalar value sampled from the interval:

$$[0, 2\pi]$$

and the target output is the cosine of that value. Small Gaussian noise was added to the inputs to make the dataset slightly noisy.

Despite having only two hidden neurons, the network was able to approximate the cosine function reasonably well. The training and testing loss decreased steadily during training, indicating that the network was learning the underlying functional relationship between the input and output.

Since cosine is a continuous function and the network has limited capacity (only two hidden neurons), the approximation is not perfect. However, the decreasing loss values and reasonably high prediction accuracy indicate that the network successfully captured the general shape of the cosine function.

4. Effect of Training Dataset Size (Deterministic Gradient Descent)

Experiments were performed with different training dataset sizes n to study how the number of training samples affects performance.

The results show that increasing the dataset size generally improves the stability of the model and provides better generalization on the test data. Larger datasets provide more information about the underlying function, allowing the network to learn more robust representations.

However, increasing the dataset size also increases computational cost per epoch. Therefore, there is a trade-off between training time and model performance.

5. Effect of Batch Size (Stochastic Gradient Descent)

Experiments were also conducted using different batch sizes m to analyze the behavior of stochastic gradient descent.

Three batch sizes were considered:

- $m = 1$ (Stochastic Gradient Descent)
- $m = 32$ (Mini-batch Gradient Descent)

- $m = 256$ (Large Mini-batch Gradient Descent)

The results demonstrate that smaller batch sizes often lead to faster convergence and better solutions. This is because stochastic gradient descent introduces noise in the gradient updates, which helps the optimization process explore the parameter space more effectively.

Larger batch sizes produce smoother gradient updates but may converge more slowly or get stuck in local minima.

6. Training and Testing Performance

Training and testing loss curves were plotted for all experiments. In general, the loss decreases over training iterations, indicating that the network is successfully learning from the data.

For the XOR problem, the network eventually achieves high classification accuracy. For the cosine approximation task, the loss steadily decreases and the predicted outputs closely follow the true cosine values.

The similarity between training and testing loss curves indicates that the model is not significantly overfitting and generalizes well to unseen data.

7. Conclusion

This experiment demonstrates that a simple feed-forward neural network trained using backpropagation can successfully learn both classification and regression tasks. The XOR experiment highlights the importance of hidden layers and nonlinear activation functions in solving non-linearly separable problems. The cosine approximation experiment demonstrates the ability of neural networks to learn continuous nonlinear functions.

The experiments with different dataset sizes and batch sizes also illustrate important optimization behaviors in gradient-based learning algorithms. Overall, the results confirm the effectiveness of backpropagation in training neural networks for different types of learning tasks.